# C PROGRAMMING — ASSIGNMENT 1

## Task 12: Re-evaluate and Adapt the Design — Day 4

Marks: 8 | Total assignment: 50 marks

### A. Impact analysis — 2 marks

The system now receives watt-hour readings every 15 minutes, which is 96 intervals per day and 35,040 intervals per non-leap year per household. It must apply a 20% surcharge to energy used between 18:00 and 22:00 and load tariff settings without recompilation. The original monthly 16-bit register assumptions and fixed #define tariff values are insufficient as the whole data model. Wider cumulative counters and timestamped interval records are needed.

### B. Evaluate original design — 2 marks

For 800,000 houses, 35,040 readings per house per year gives 28,032,000,000 interval readings per year. This exceeds a 32-bit unsigned count (4,294,967,295) if represented as one global interval count. A 64-bit unsigned integer is appropriate for annual aggregate counts and watt-hour totals. Individual interval values may fit in uint32_t, but cumulative annual totals should use uint64_t or checked int64_t. The 16-bit meter register is not adequate as a long-term cumulative watt-hour total.

### C. Adaptation/redesign — 2 marks

Use a timestamp for each interval, a wide integer watt-hour/energy counter, and tariff settings loaded and validated from a configuration file at startup. Apply the 20% peak surcharge only to interval energy during 18:00–22:00. To avoid fractional paise, calculate the base charge in paise and represent the 20% surcharge as an integer numerator/denominator; define a rounding policy, such as nearest paise. Production code should also handle time zones, daylight-saving changes where relevant, missing intervals and meter resets.

### D. Evidence-based defence — 2 marks

The annual interval count calculation is 35,040 × 800,000 = 28,032,000,000, requiring more than 32 bits. A uint64_t count has ample range. Reading tariff configuration costs O(t) time at startup for t values and O(t) storage; in exchange, tariff revisions do not require recompilation. The design should be accepted only after tests for peak and off-peak intervals, day boundaries, missing data, tariff-file errors, large totals and rounding boundaries.

### Design pseudocode

LOAD and validate tariff configuration

FOR each interval record:

validate timestamp and watt-hour value

determine units/energy consumed in interval

IF local time is in [18:00, 22:00):

charge = base_charge × 120 / 100, rounded by stated rule

ELSE:

charge = base_charge

accumulate charge in checked 64-bit integer paise

REPORT total, base energy, peak surcharge and validation errors

### C Source Code / Implementation

```
/* Task 12: illustrative interval tariff calculation.
   Amounts are integer paise; 20% surcharge is rounded to nearest paise.
   This example accepts the time window as a caller-supplied boolean.
*/
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdbool.h>

static int64_t apply_peak_rate(int64_t base_paise, bool peak) {
    if (base_paise < 0) return -1;
    if (!peak) return base_paise;
    /* nearest-paise rounding for 120%: (base*120 + 50)/100 */
    if (base_paise > (INT64_MAX - 50) / 120) return -1;
    return (base_paise * 120 + 50) / 100;
}
int main(void) {
    const uint64_t houses = UINT64_C(800000);
    const uint64_t intervals_per_house_year = UINT64_C(35040);
    uint64_t annual_intervals = houses * intervals_per_house_year;
    int64_t off_peak = apply_peak_rate(225, false);
```

```
    int64_t peak = apply_peak_rate(225, true);
    printf("Annual intervals across district: %" PRIu64 "\n", annual_intervals);
    printf("Base charge: %lld paise; peak charge: %lld paise\n",
           (long long)off_peak, (long long)peak);
    printf("Peak example: Rs %lld.%02lld\n",
           (long long)(peak / 100), (long long)(peak % 100));
    return 0;
}
```

## Submission note

Compile and run the code on your own system, save actual terminal output/screenshots, and include them in your report. Review the code and be prepared to explain it during the viva. Any machine-specific measurements must come from your own execution.